

Part 9: “Swarm robotics 2.0”

Collective Robotics

Prof. Dr. Javad Ghofrani



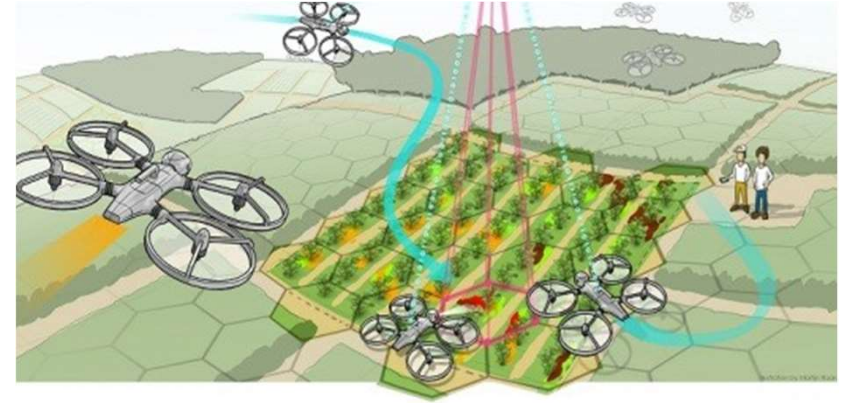
**Hochschule
Bonn-Rhein-Sieg**
University of Applied Sciences

“Collective Robotics 2.0”

Get out of the lab

Error detection and security

Interfacing robots and robots as interface

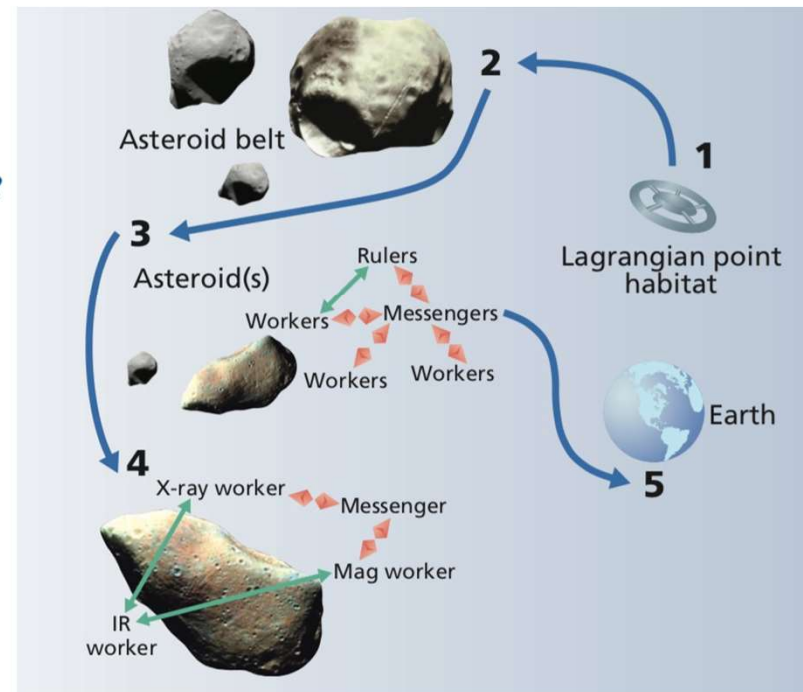




Collective robotics
as field robotics?

NASA's Swarm Missions: The Challenge of Building Autonomous Software

In future space missions, thousands of tiny spacecraft will cooperate to explore the solar system. Providing the required autonomy will take systems and software where no one has gone before.



Collective robotics as field robotics?



<http://www.subcultron.eu>

examples

subCULTron (underwater robotics)

aquatic surface robots (Duarte et al., 2016)



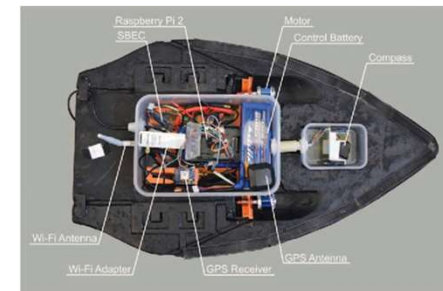
<http://journals.plos.org/plosone/article?id=10.1371/journal.pone.0151834>

Getting out of the lab - a swarm on the ocean

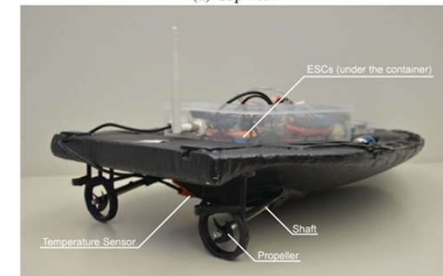
Despite the potential of swarm robotics for real-world tasks, no demonstrations of swarm behaviors outside of controlled laboratory conditions have been performed so far [...] all real-robot experiments have, in fact, been performed in controlled laboratory environments. (Duarte et al (2016))

Evolutionary swarm robotics out of the lab

- 10 aquatic surface robots
- differential drive mono-hull boat
- Raspberry Pi 2, IEEE 802.11g WiFi, GPS, and a compass
- experiments include:
 - homing (navigation to waypoints), dispersion, aggregation, and area monitoring



(a) Top view



(b) Side view

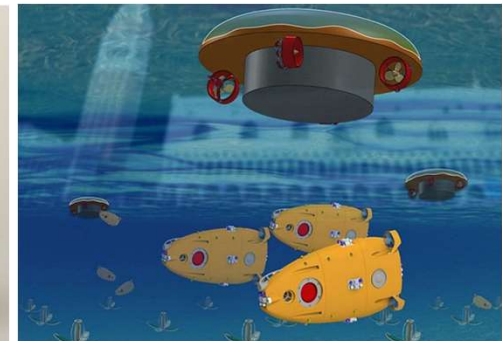
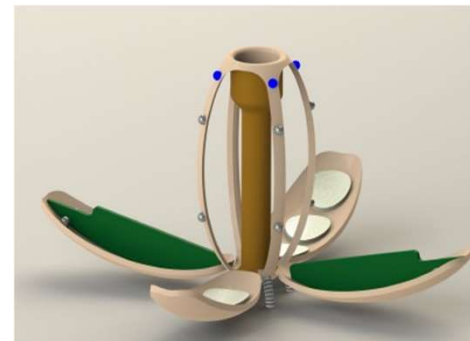
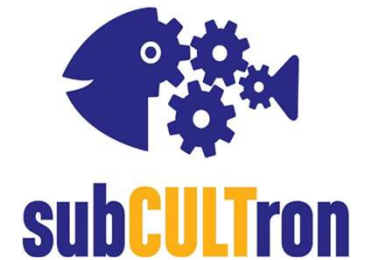


Duarte, M., Costa, V., Gomes, J., Rodrigues, T., Silva, F., Oliveira, S. M., et al. (2016). Evolution of collective behaviors for a real swarm of aquatic surface robots. *PLoS One*, 11(3), 1–25. <https://doi.org/10.1371/journal.pone.0151834>.

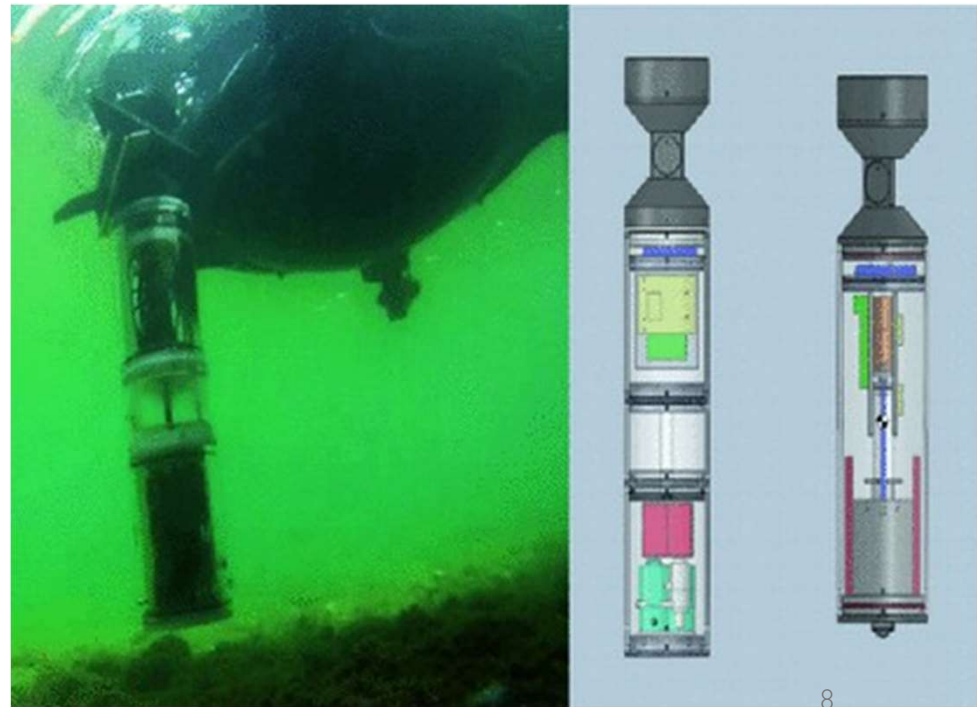
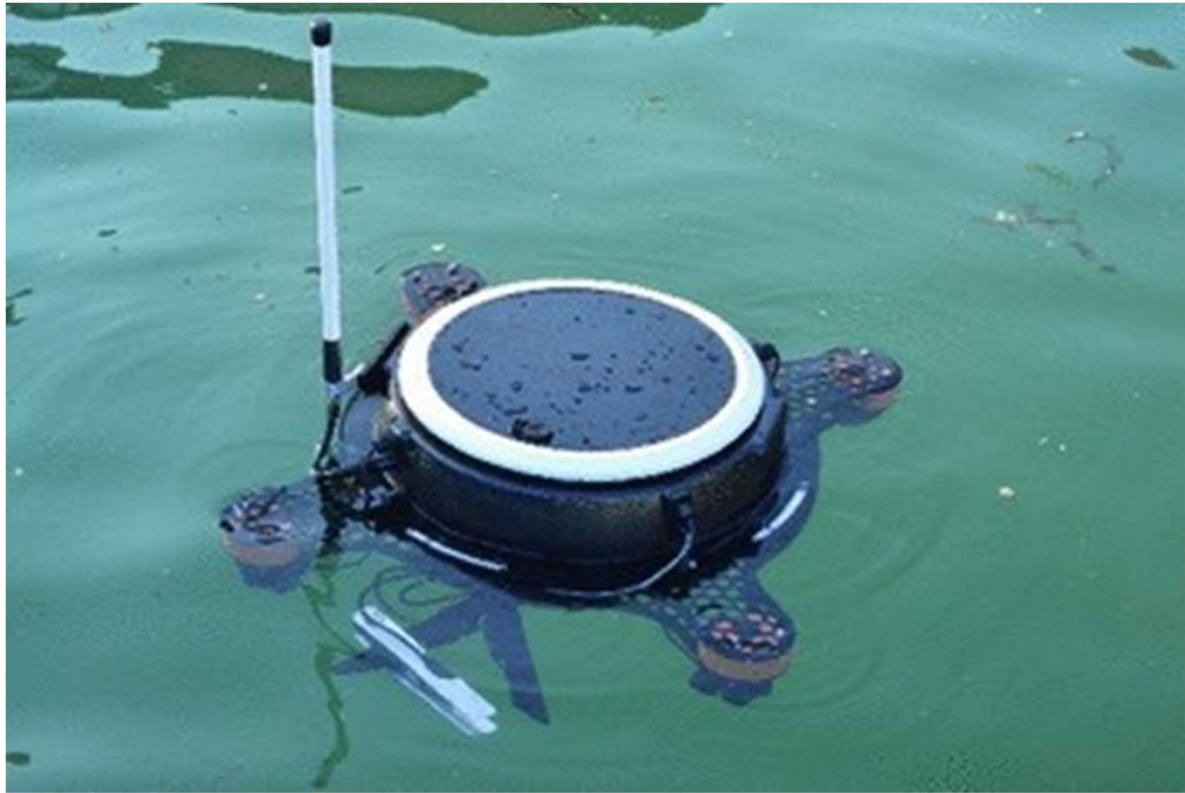
Getting out of the lab - a swarm on the ocean

Project subCULTron

- autonomous underwater vehicles (AUV)
- 3 types:
- sea-ground - artificial “mussels” (<15m depth, acoustic com.)
- water-surface - artificial “lily pads” (GPS, GSM, solar panel, docking)
- in-between - artificial “fish” (sensors: pressure, temperature, com via light)
- monitoring water quality in the lagoon of Venice
- up to 100 robots



Project subCULTron



Project subCULTron



Robustness 2.0: delivering true robustness

Robustness 2.0: delivering true robustness

Prevailing challenges of robustness & fault-tolerance in collective robotics

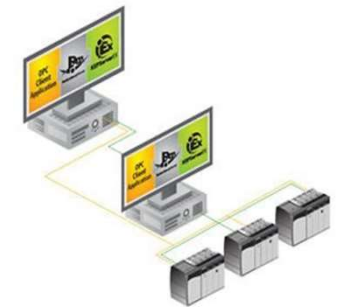
High redundancy (many robots of the same kind) and flexibility (any robot can complete any task) are a good start for robustness

This is hardware redundancy,

what about software redundancy? all robots run the same program, one bug can kill them all

What if one robot fails but it keeps sending wrong messages that disturb the whole system? Can the swarm detect that and turn it off or ignore it?

Say the system is generally robust against robots failing; what if a decreased swarm size or swarm density requires adapted control parameters?



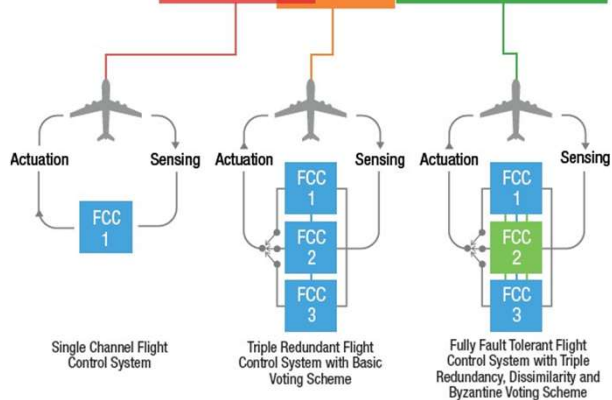
Robustness? Learning from the aviation industry

robustness as fault tolerance in aviation

strict rules: design assurance level (DAL)

single-board computers require DAL A: less than 1 in 10^{-9} probability of failure per flight hour

0.000000001/Flight Hour



Airbus A330, avionics bay

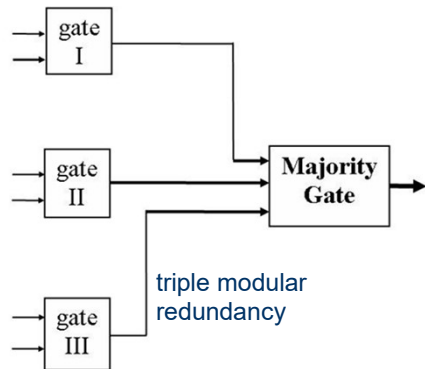
<https://www.flickr.com/photos/26436024@N06/6853053865/>

dissimilar redundancy

- If a monolithic system cannot be made fault-tolerant enough, then combine several “unreliable” modules that differ as much as possible from each other.
- e.g., Airbus A320:
- 5 dissimilar computers running 4 dissimilar software packages (OS, application)



Voting schemes



vote tally for option y is

$$w = \sum_{i \in \{i | y = x_i\}} v_i$$

say option y has the highest vote tally
(without loss of generality)

voting schemes

consensus voting: choose option y

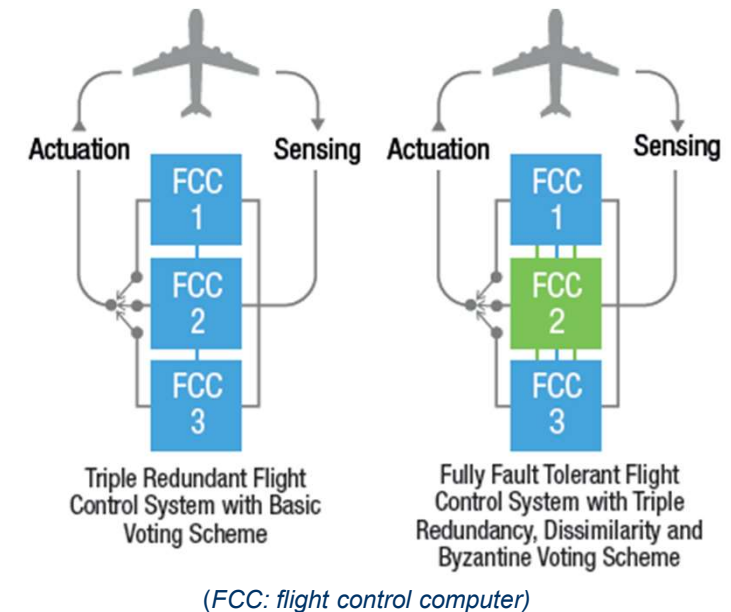
majority voting: choose option y if

$$w > \frac{1}{2} \sum_i v_i$$

Byzantine voting: choose option y if

$$w > \frac{2}{3} \sum_i v_i$$

(<1/3 of votes can be associated with faulty nodes)



n voters

with voting weights v_i

voting for option $x_1, x_2, \dots, x_n \in \{0, 1, \dots, m\}$

Software redundancy: N-version programming

Aka multiple-version dissimilar software (1977 by Liming Chen and Algirdas Avizienis)

method from **software engineering**: programming **independently**
multiple **functionally-equivalent** programs from the same specification

N human **groups** develop programs independently (best case: without knowing of each other,
using different programming languages, different algorithms)

an **N-version execution environment** has to administrate the N programs and select whose output should be passed through (majority etc.)

problems:

Are bugs **statistically independent**?

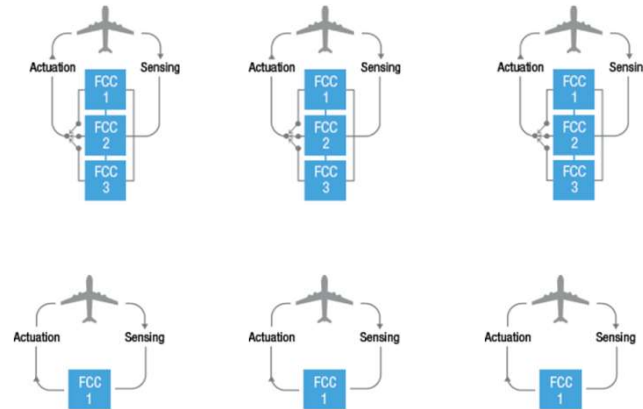
Or do different teams make **similar mistakes**?



N-version programming in robot swarms

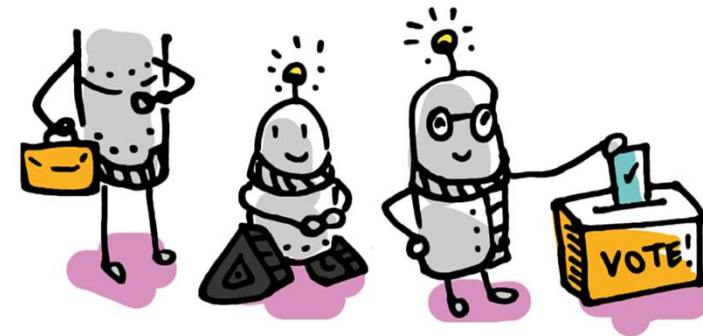
2 options for applying *N*-version programming in swarms:

- (A) implementation on **microscopic** level (single robots): each robot runs **all** *N* controllers and onboard its own *N*-version execution environment
- (B) implementation on **macroscopic** level (robot swarm):
N sub-populations of robots running **one** of *N* different controllers each



in the case of B, 2 more options:

- (B1) no explicit *N*-version execution environment,
hope for effect **without voting**
- (B2) explicit *N*-version execution environment
implemented via communication,
robots **coordinate by voting** with their neighbors



Fault tolerance by explicitly detecting errors

idea of **fault tolerance**:

prevent **total** breakdowns requires:

- (1) no single point of failure
- (2) fault isolation: failure detection
- (3) fault containment / mitigation / recovery

graceful degradation

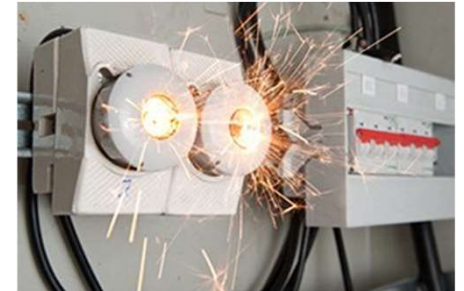
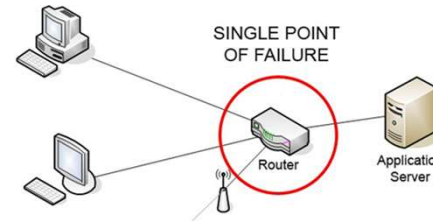
maintain functionality even if parts break down, but possibly with reduced throughput / efficiency

idea for a fault-tolerance system approach

in **collective robotics**:

actively **detect** and accommodate **faults**

robots monitor each other, detect faults in others, and vote to exclude swarm members



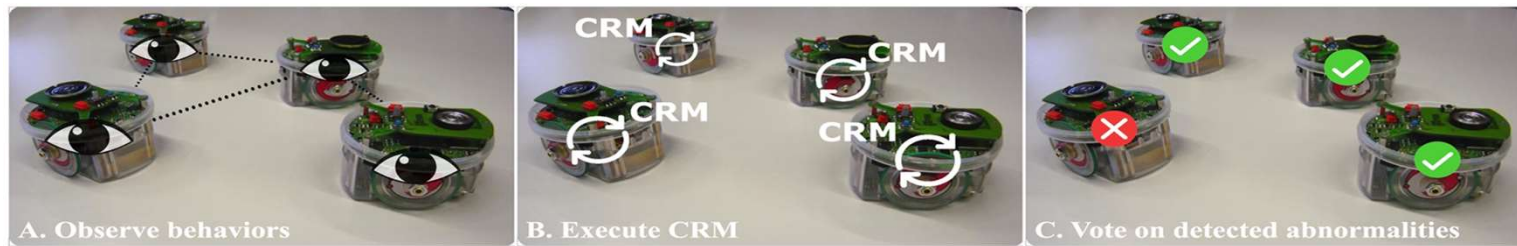
© Farbentek, Shutterstock



Decentralized fault detection (overview)

actively detect and accommodate faults

generic fault-detection system for robot swarms, learning online



- (A) robots observe the behavior of their neighbors
and estimate the corresponding **behavioral features** (see next slide)
- (B) each robot runs a cross regulation model (immune system-inspired algorithm,
see next slide) to **detect abnormal behaviors**
- (C) robots of the swarm form **voting coalitions** on the detected abnormalities to consolidate
their individual decisions

<http://journals.plos.org/plosone/article?id=10.1371/journal.pone.0182058>

<https://doi.org/10.1109/TEVC.2009.2017516>

D. Tarapore,
A. L. Christensen,
J. Timmis (2017)
*Generic, scalable and
decentralized fault detection
for robot swarms.*
PLOS ONE 12(8): e0182058.

Decentralized fault detection (behavioral features)

behavioral features

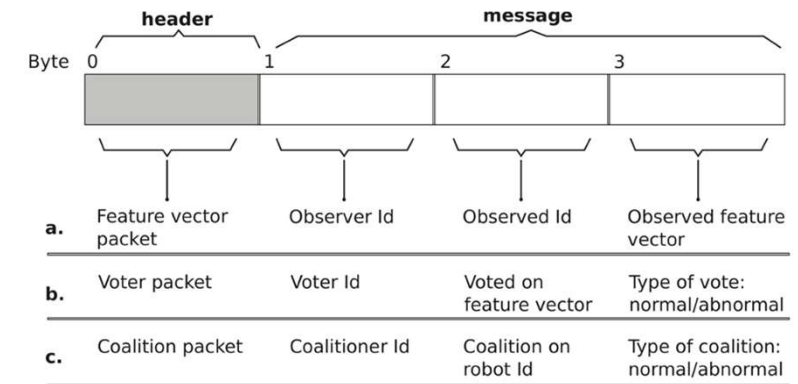
a human designer **cleverly selects** so-called behavioral features: a vector of **observable aspects** of the robot behavior that may help to **classify and distinguish** behaviors; examples are:

- robot has at least one **close-by / far neighbor**
- robot has **turned / moved** recently
- robot changes its heading in **presence of close-by neighboring** robots

vectors can easily be averaged, **communicated**, and **compared**; requires **thresholds** to define whether two vectors still count as same

for **voting**, each robot i **communicates** its behavioral vector about robot j to its neighbors

each robot then gets a vector assigned by **majority vote**



Decentralized fault detection (CRM)

Cross regulation model (CRM)

roughly inspired by the immune system
imitates the population dynamics of immune cells CRM is a system of ODE

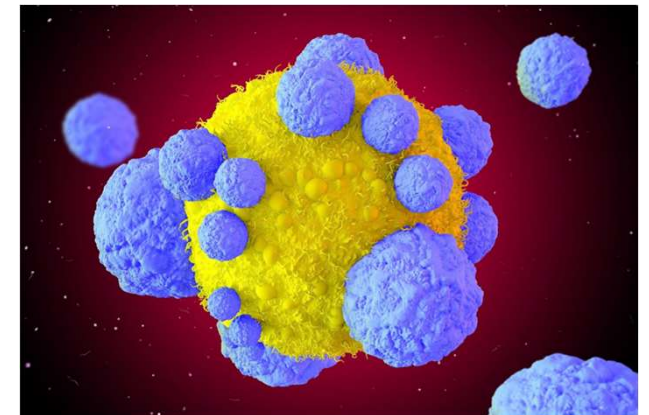
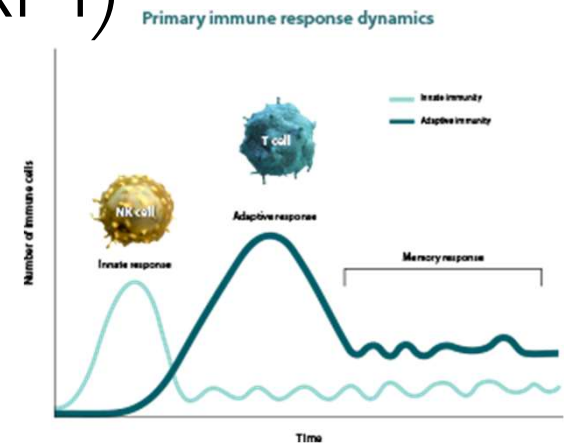
system is **bistable** with two possible outcomes:

- immune response (abnormal behavior requires action)
- or tolerant response (normal behavior, no action required)

integration over time of how often a robot was labeled abnormal

behaviors performed by **most robots** are to be treated as **normal** behaviors exhibited by **fewer robots** are to be treated as **abnormal**

tradeoff between latency and false positives
balances cost of “too late” vs “wrong accusation”



Roger Harris/Science Photo Library/Getty Images

<https://doi.org/10.1109/TEVC.2009.2017516>

<http://journals.plos.org/plosone/article?id=10.1371/journal.pone.0182058>

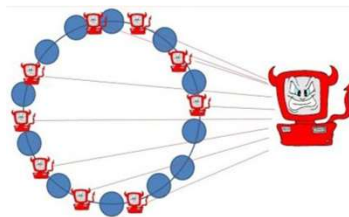
“Robot swarms need blockchain technology”

Blockchain & Ethereum

Ethereum

a decentralized, open-source blockchain

not limited to financial transactions but e.g.
exchanging snippets of code blockchain-based
smart contracts



ethereum

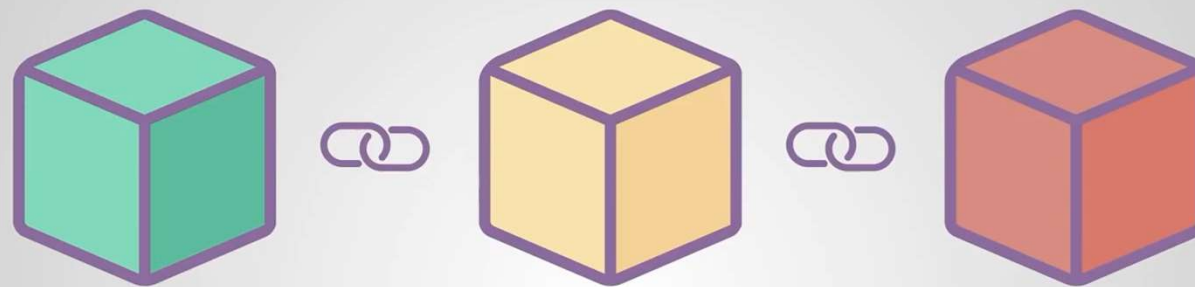
V. Strobel, E. Castello, M. Dorigo
Robot swarms need blockchain technology
Front. Robot. AI, May 2020
<https://doi.org/10.3389/frobt.2020.00054>



Motivation

Byzantine robots: umbrella term for robots showing unintended or inconsistent behavior (Byzantine Generals Problem)

Sybil attacks: corrupting a reputation system by creating large numbers of pseudonymous identities

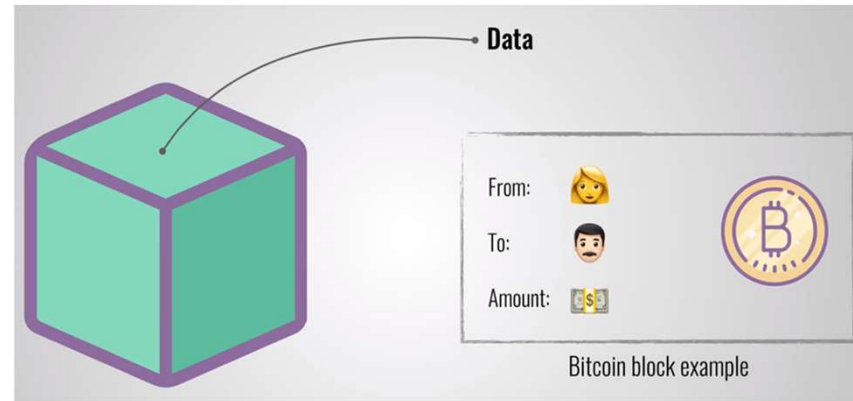
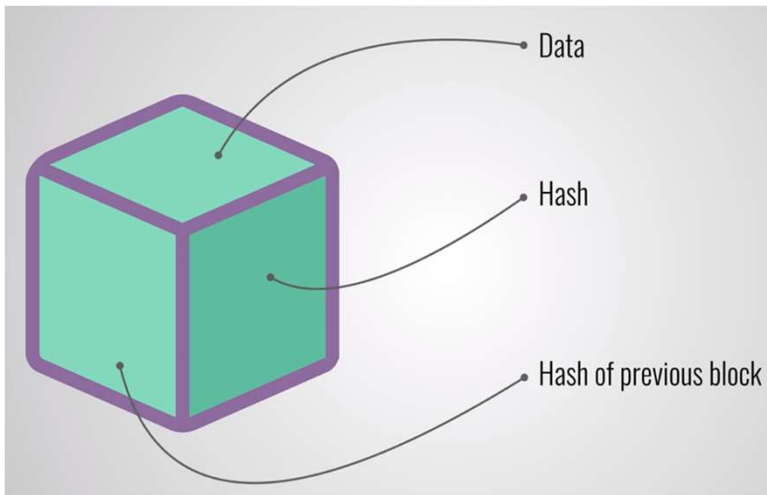
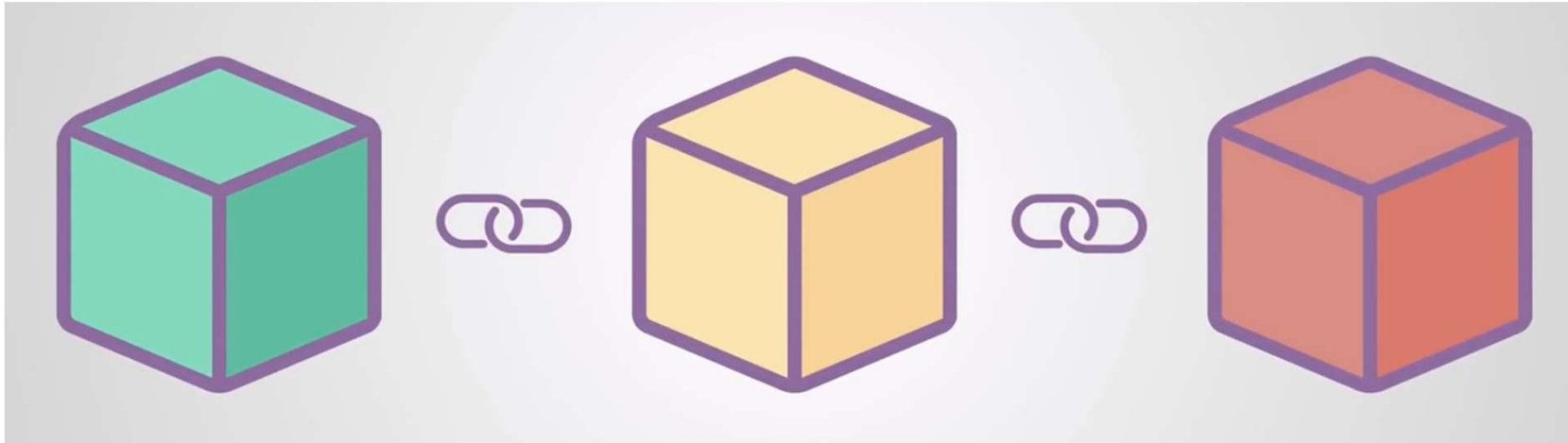


Blockchain

— *Simply explained* —

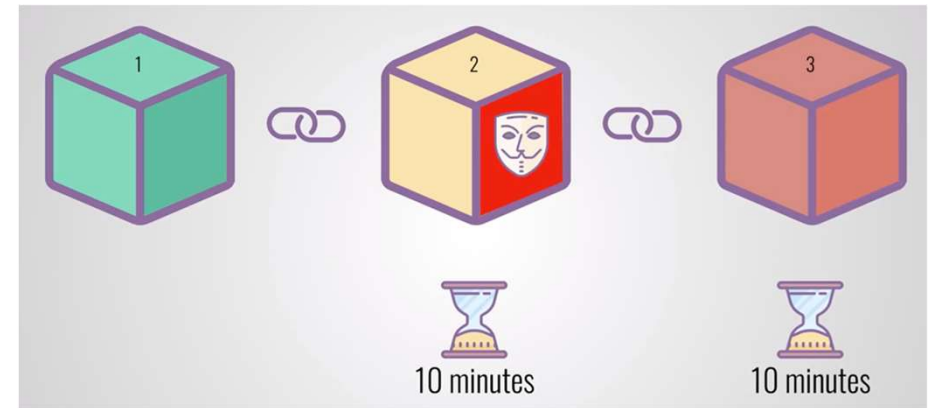
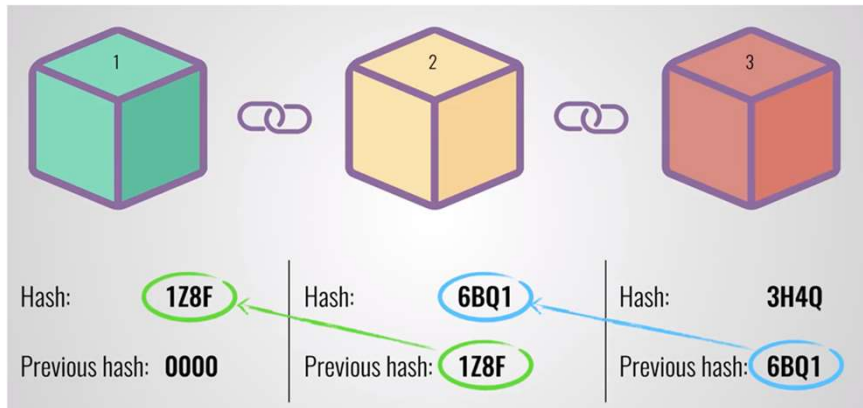
https://www.youtube.com/watch?v=SSo_ElwHSd4

Blockchain (1/2)

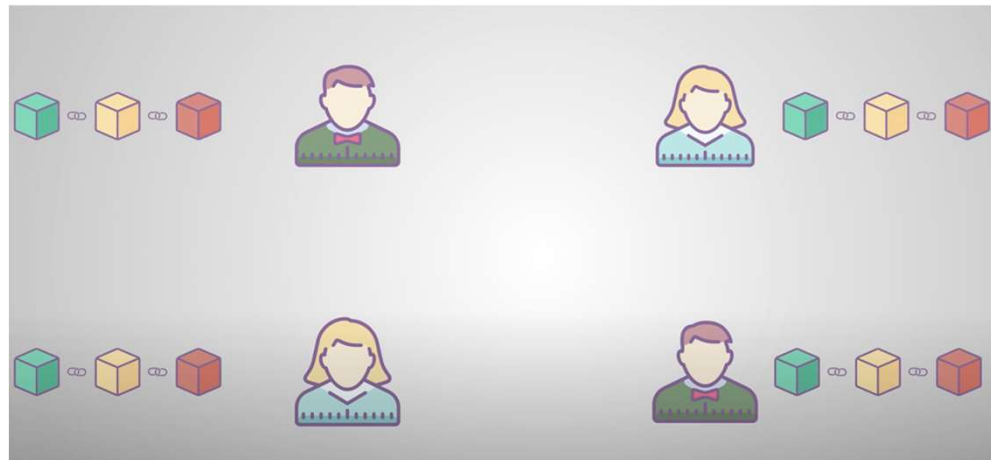


https://www.youtube.com/watch?v=SSo_ElwHSd4

Blockchain (2/2)



https://www.youtube.com/watch?v=SSo_ElwHSd4



Blockchain in collective robotics

advantages

contract conditions are transparent

existing votes cannot be manipulated or discarded

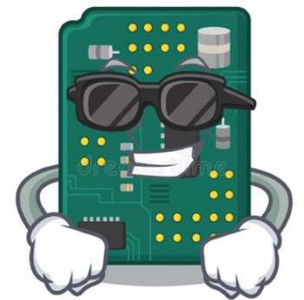
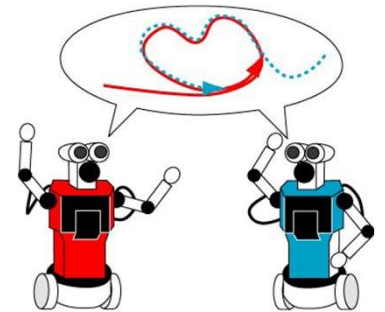
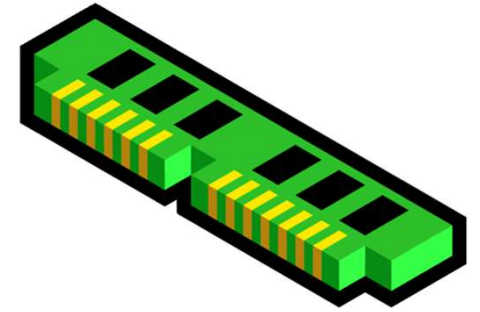
challenges

computational demands due to computational puzzle, mining

conflicting blockchain versions (forks)

memory & communication demands: size of total blockchain

(size per transaction: 150 Bytes, observed average blockchain sizes in experiments: 6.8 MB)

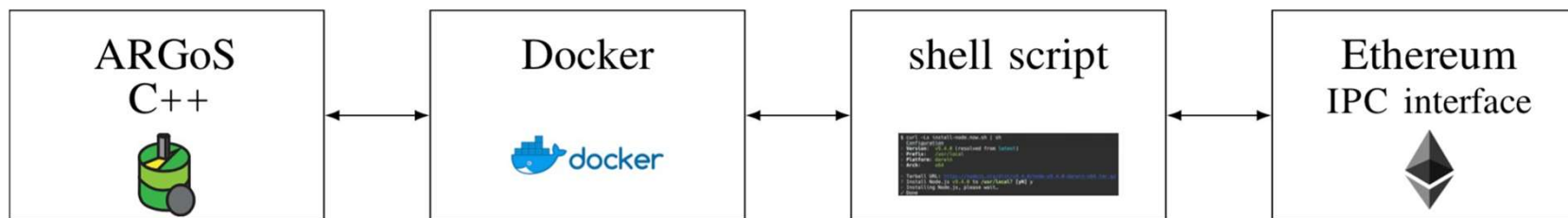
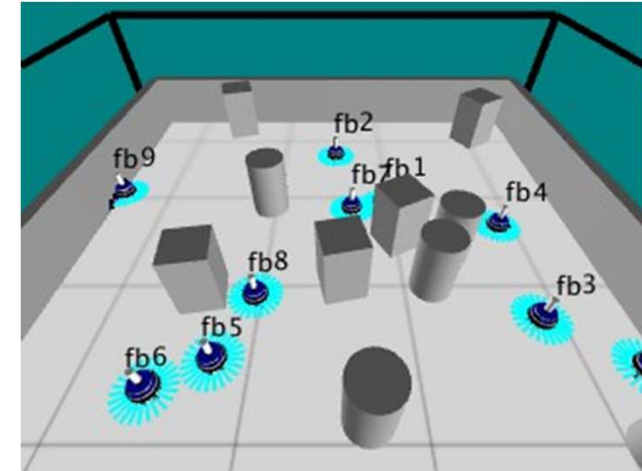


Simulation

This is based on a simulation-only study

ARGoS - there is a plugin for blockchain

Everything is run on a server supported by a Docker for each robot



Blockchain - general foundation



ethereum

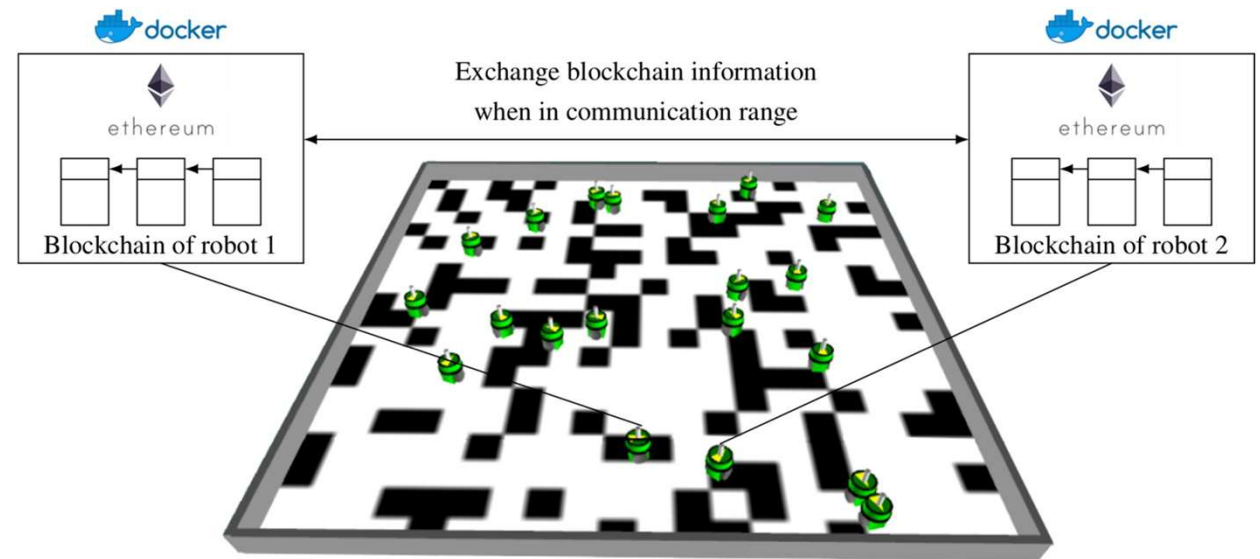
Transactions are signed by the sender using a private key
to add a transaction to the blockchain requires consensus

Consensus protocol requires Proof-of-Work (PoW) e.g., finding a hash value below a target value process of solving this puzzle is called mining
(alternatives: Proof-of-Stake, Proof-of-Sensing, Proof-of-Physical-Work)

The blockchain is stored distributed over all robots (ideal case is that all robots have an identical local copy)

Due to delays (e.g., several connected components) participants can have conflicting blockchain versions (forks)

Task: collective decision making



Benchmark task

Collective decision making: agree
on a frequency of white tiles 400 tiles

white tile frequency 75%

robots estimate individually: $\rho' = \frac{W}{W + B}$

Collective decision making with blockchain (1/2)

Each robot keeps a local copy of the blockchain Each robot mines the full time,
Once it solves a block it is rewarded by 5 ether

exploration phases of 45 seconds,
each robot sends its sensor reading to the smart contract

to store a sensor reading in the blockchain a robot:

1. creates a blockchain transaction
2. adds a deposit of q ether (q is design parameter)
3. signs the transaction
4. disseminates this transaction among its neighbors



Collective decision making with blockchain (2/2)

The current smart contract estimate and the individual estimate ρ' are compared:

- (a) difference $|\rho - \rho'| > \varepsilon$, too big: discarded as **outlier**
- (b) difference $|\rho - \rho'| \leq \varepsilon$, fine: accepted as **inlier**

Parameter ε is a **design parameter**, here $\varepsilon = 0.2$

robots submitting an inlier get a **reward** of r_t ether (depending on total number of inliers at time t etc.)

Byzantine robots try to **sabotage** the process by sending wrong estimates of $\rho' = 0$
(true frequency of white tiles 75%)

Sybil attack: still try to sabotage the process by sending estimates of $\rho' = 0$
+ try to send as **many messages** as possible by
creating new identities at each time step

Baseline: Linear Consensus Protocol (LCP)

To compare to a simple default algorithm
we use the **linear consensus protocol (LCP)**

each robot i in phase m keeps a frequency estimate $\hat{\rho}_{i,m}$
in the 1st phase ($m=0$) it is set to the measured value $\hat{\rho}_{i,0} = \rho'_{i,0}$ a
robot's estimate is then **updated** in each exploration phase m by
incorporating the frequency estimates of its neighbors $\mathcal{N}_i$:

$$\hat{\rho}_{i,m} = w_{ii}\rho'_{i,m} + \sum_{j \in \mathcal{N}_i} w_{ij}\hat{\rho}_{j,m-1}$$

with weight factor

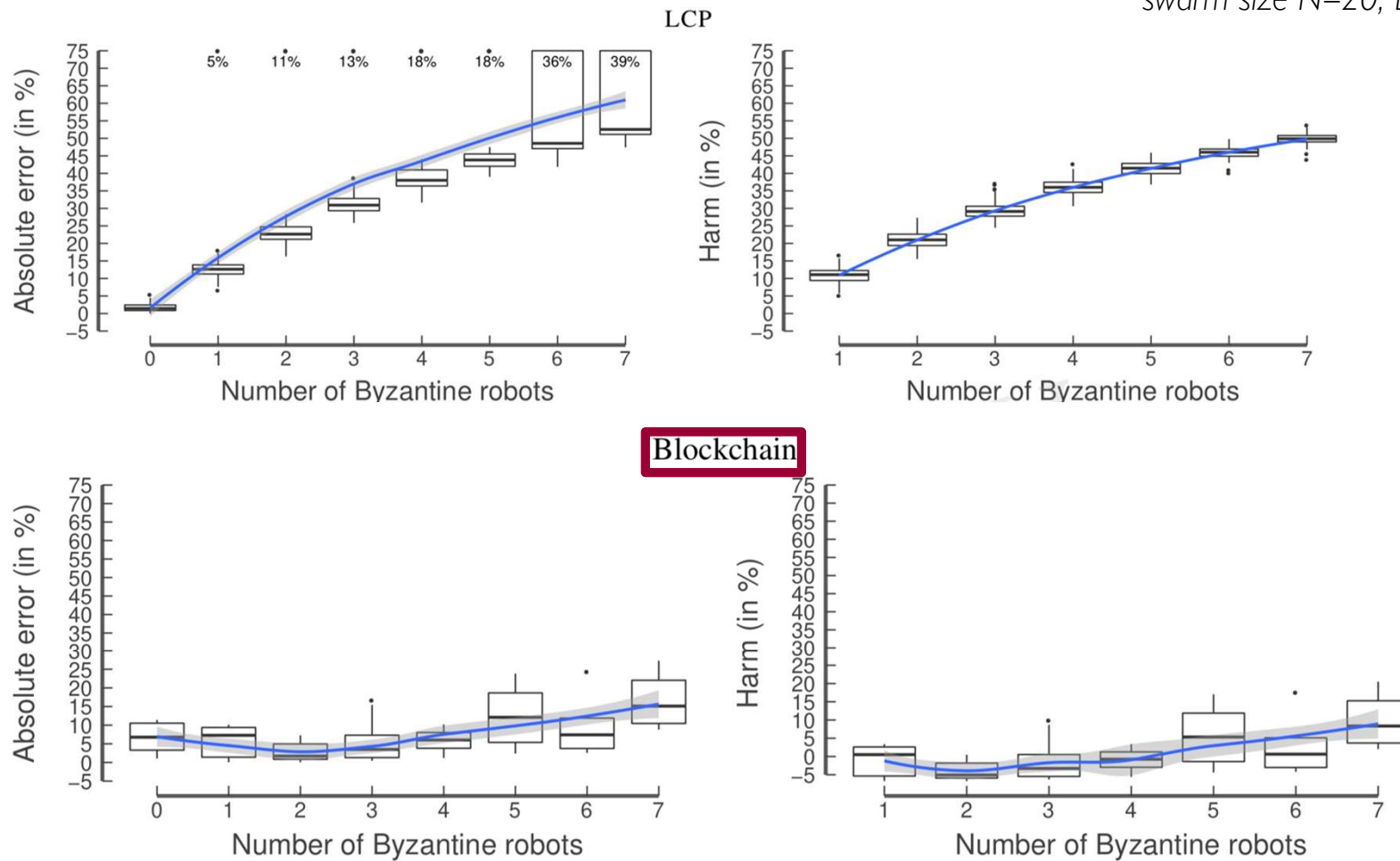
$$w_{ii} = w_{ij} = \frac{1}{|\mathcal{N}_i| + 1}$$

for comparison we define **harm**:

difference between the average absolute **error of non-Byzantine robots** and the median average
absolute **error for experiments without Byzantine robots**

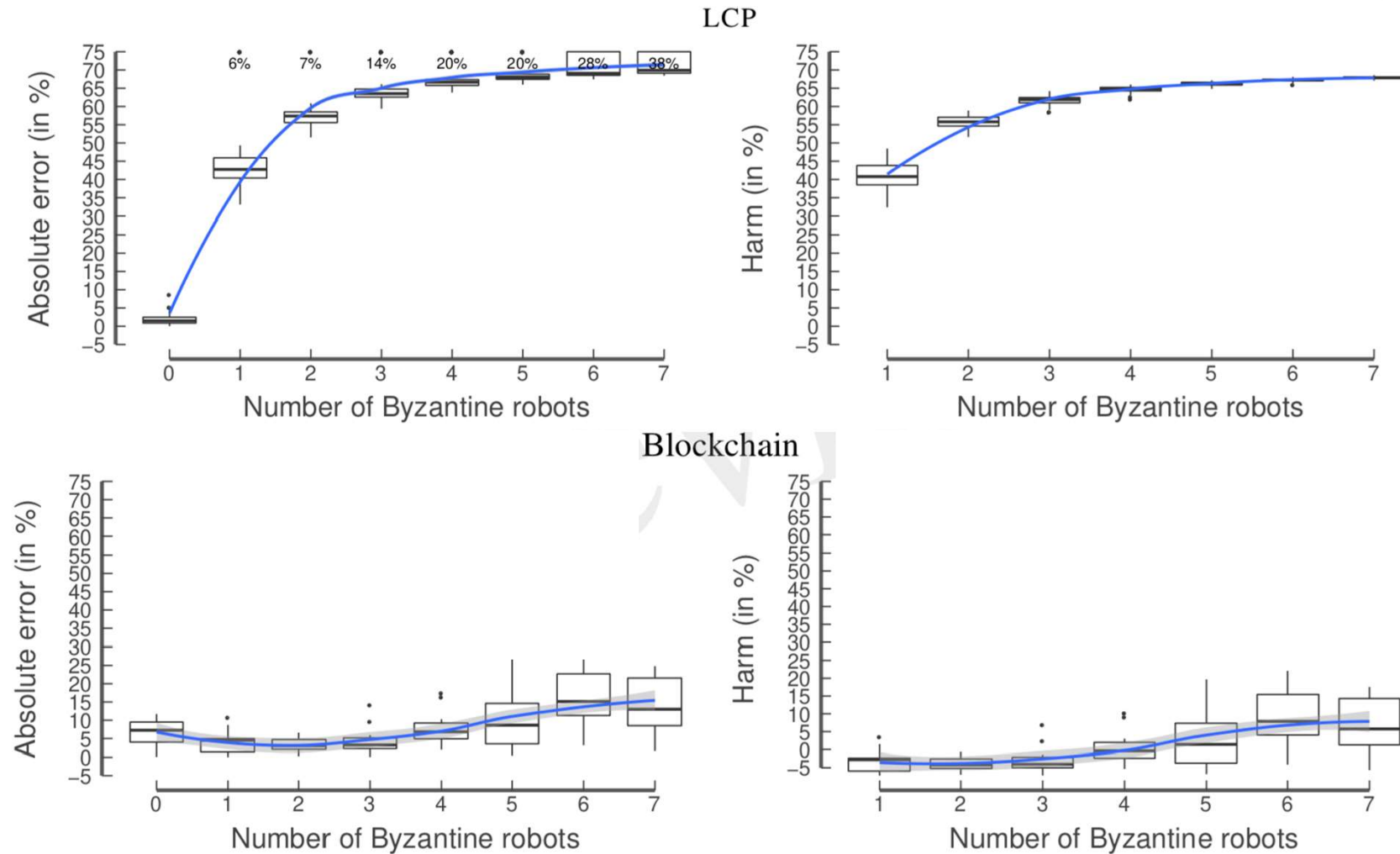
Results: Comparison Influence Byzantine Robots

swarm size $N=20$, Byzantine robots



Results: Comparison Influence Sybil Attack

swarm size $N=20$, Sybil attack



Remaining Weaknesses of Blockchain Approach

There are not too many options left for attackers.

51% attack

group of miners (robots) controlling **more than 50%** of the network's (non-Byzantine robots) mining hashrate / **computing power**
(e.g., attacking robots with uplink to the cloud, big robot with a lot computing power)

parameterization of artificial economy

especially against Sybil attacks, **costs of messaging and rewards for mining**
(in currency ether) need to be carefully parameterized

other limitations

increased **latencies** due to complex algorithmic approach

storage requirements: approx. 30 MB generated within 24h, robots have 16 GB mem.

battery drain due to PoW



Interfacing robots and robots as interface

Interfacing robots and robots as interface

(1) How can we **interface swarms of robots**? use case: a flock of quadcopters need to be navigated, can a human operator control one of them and the swarm follows?



(2) Can we use **robot swarms as interface** for other uses?
use case: motion and/or position of robots signals system states, a user may move robots around as input



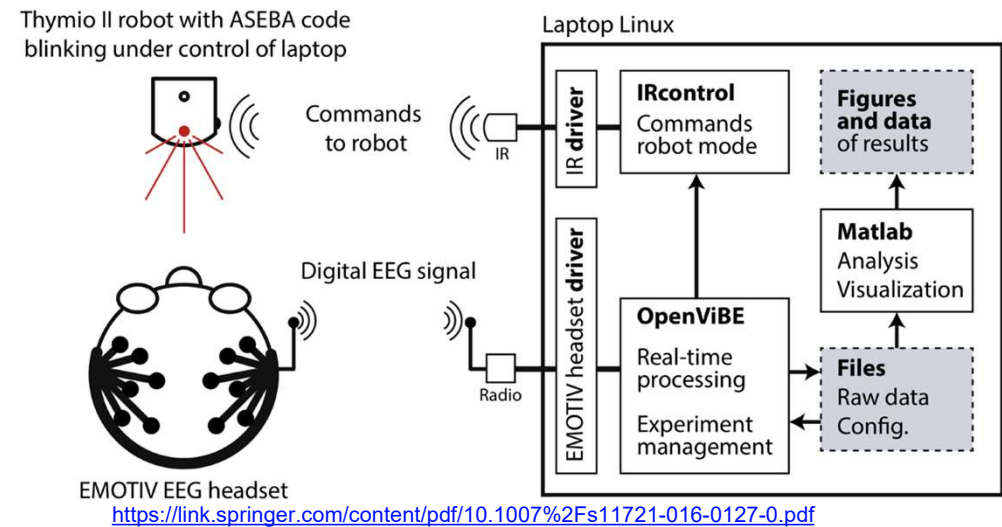
Interfacing a robot swarm with EEG

Implementation

- ▶ EEG as implicit communication channel for proximal interaction (Mondada et al.)
- ▶ using electroencephalography (EEG)
- ▶ exploiting steady-state visually evoked potential (SSVEP): neural response to a regularly blinking visual stimulus



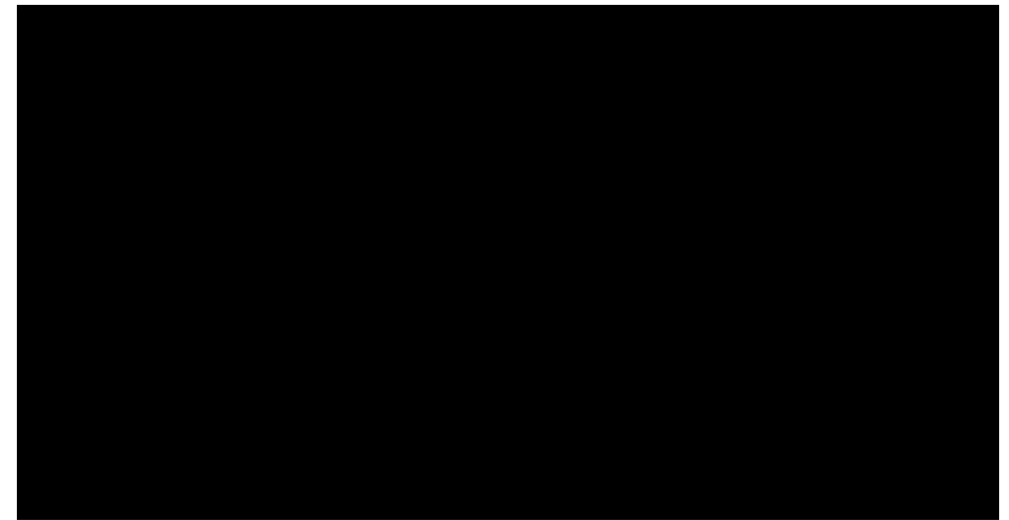
<https://infuu.de/high-tech/188/emotiv-new-epoc-eeeg-neuroheadset-bci>



Interfacing a robot swarm: robot selection

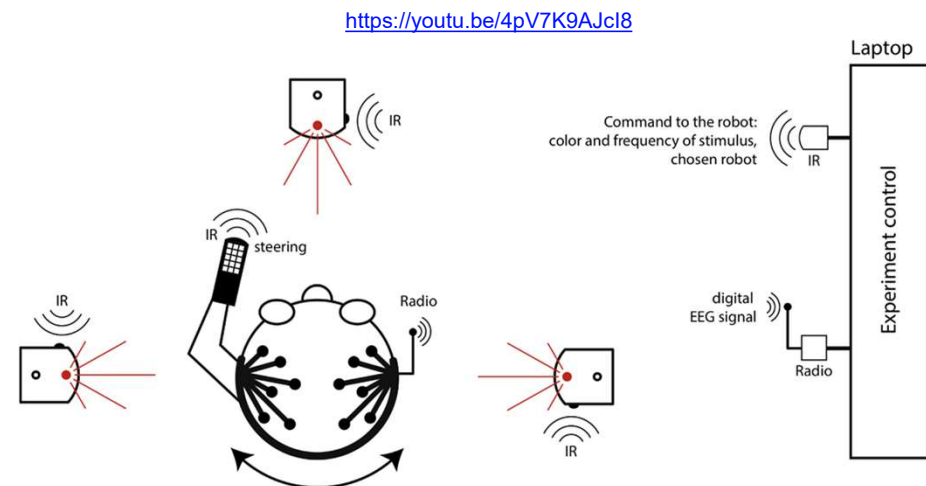
Concept

- ▶ 3 Thymio robots **blinking** with 8, 10, and 12 Hz
- ▶ selection of robot:
subject **looks at** the robot to be controlled,
system detects via SSVEP which robot is selected
- ▶ actual command is sent via **remote** using IR

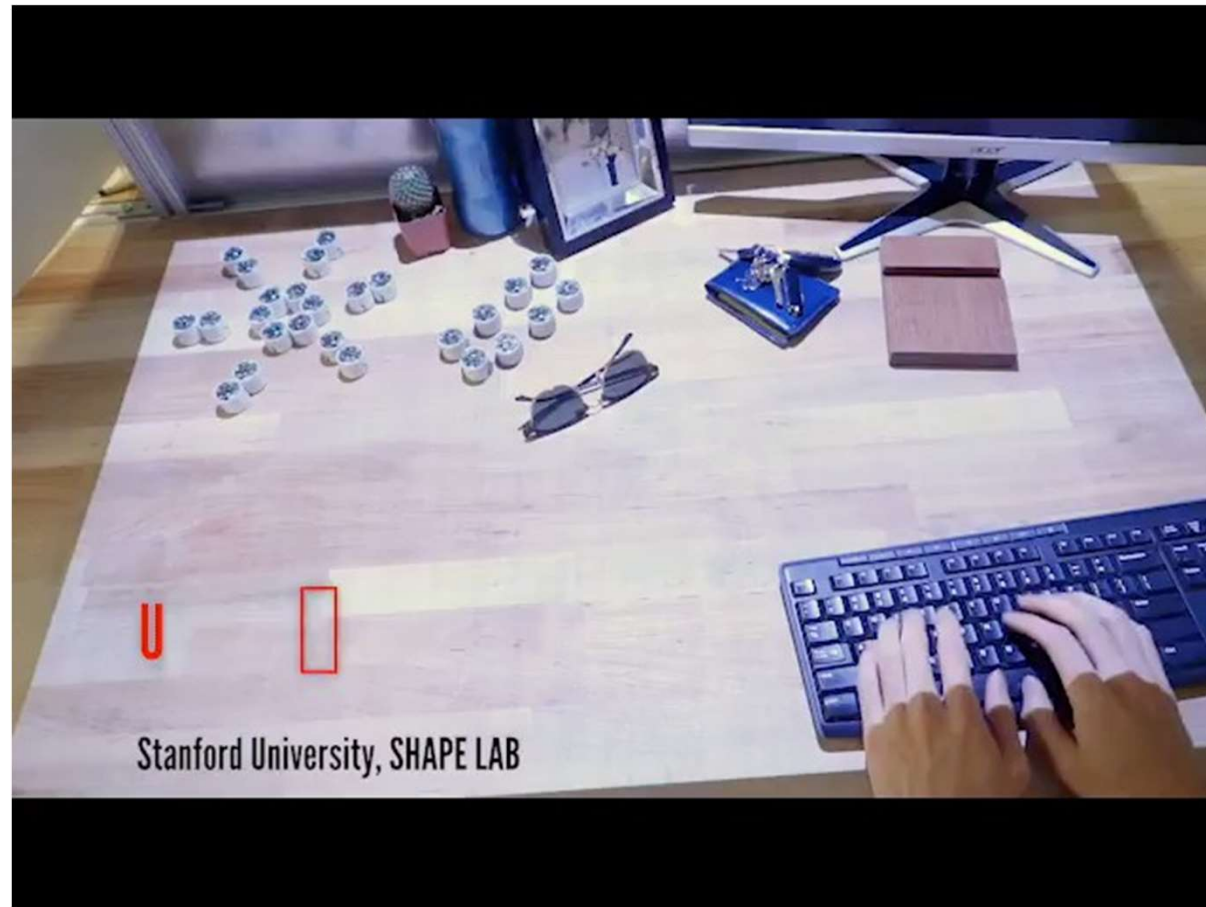


Limitations & future options

- ▶ scalability limited by **blinking frequencies** that can be distinguished
- ▶ limited to **proximal** interaction (line of sight etc.)
- ▶ extracting SSVEP response from EEG signal: very high variability
- ▶ future option:
allow for **initial training phase** and use
machine learning to adapt to the subject



Collective robotics as interface?



<https://www.youtube.com/watch?v=oT7theBRBzl>

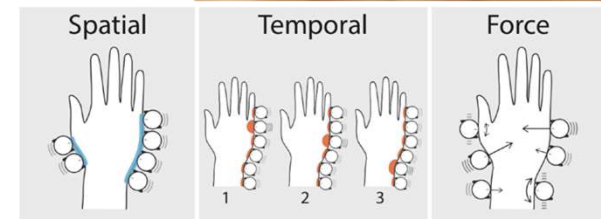
“SwarmHaptics”: haptic display with swarm robots

Implementation

- “Zooids” robots
- projector-based tracking
- magnets and ferromagnetic surface for increased force output (0.9 N)

Haptic parameters & types of coordinations

- force: translation (pushing) and shear (turning)
- contact location: finger, hand, wrist, elbow, etc.
- tactile (stimulating skin)
vs kinesthetic (moving joint angles)
- spatial, temporal, and force patterns



<http://shape.stanford.edu/research/2019-SwarmHaptics/SwarmHaptics.pdf>

“SwarmHaptics”: user perception

Results

- measured effects on emotion (arousal, valence, dominance)
- measured effects on human-robot interaction (HRI, anthropomorphism, likeability, safety)
- measured effects on perceived urgency and willingness to attend

Future

- real-time multi-point haptic feedback for interactive applications

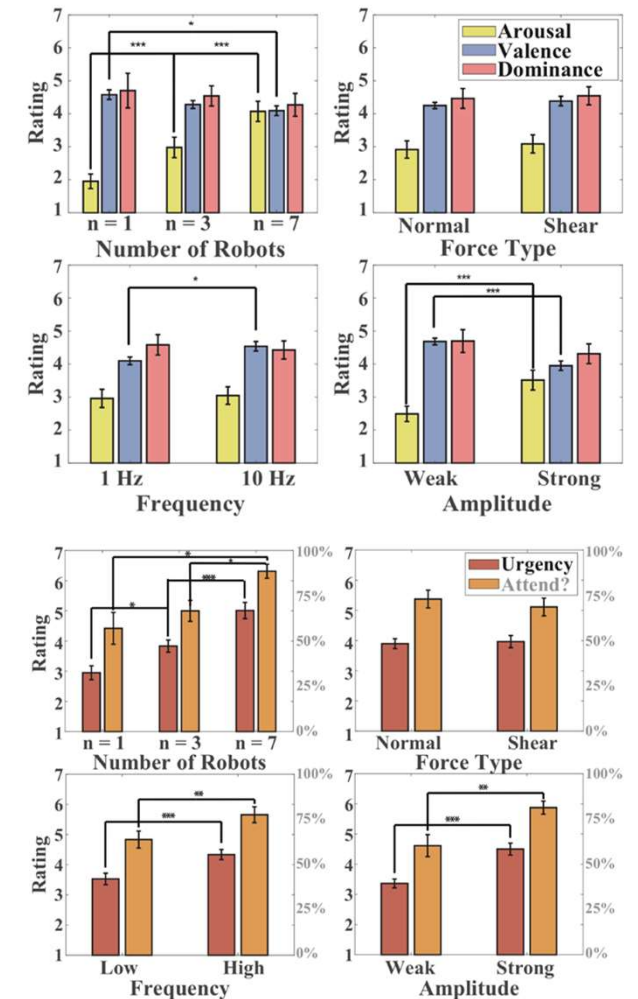


Figure 10: Effect of haptic parameters on perceived urgency and willingness to attend.

Collective robotics in construction

2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
Macau, China, November 4-8, 2019

Self-organized adaptive paths in multi-robot manufacturing: reconfigurable and pattern-independent fibre deployment*

Catriona Eschke^{1,2}, Mary Katherine Heinrich², Mostafa Wahby², and Heiko Hamann²

SELF-ORGANIZED ADAPTIVE PATHS IN MULTI-ROBOT MANUFACTURING: RECONFIGURABLE AND PATTERN-INDEPENDENT FIBRE DEPLOYMENT

CATRIONA ESCHKE, MARY KATHERINE HEINRICH,
MOSTAFA WAHBY, AND HEIKO HAMANN

robot systems for autonomous construction and scalability. Swarm exploits robot interactions and a multi-robot swarm collectively constructs a shape comparable robot could achieve. A new concept of swarm construction uses continuous building material. In its use of braiding techniques, it employs fibres that potentially allow for flexible building blocks. To achieve this, we restrict ourselves to a decentralized approach to task allocation, and the challenges are the local coordination of the braiding scheme at runtime. We approach this in multi-robot experiments and branching of the braid. In addition, we implement an open system—that is, the braiding process on the fly.

building material that potentially allows for infinitely many shapes and forms. We use mobile robots to braid fibres. Although some self-organization has been implemented in our previous work with line-following robots on predefined tracks [7], swarm robotics approaches to fibre deployment are generally underexplored. Our previous work [7] did not include autonomous initialization, adaptive paths, or reconfigurability during runtime, each of which are important for a swarm approach and are addressed here. Scalability will be a relevant issue in the manufacture even of moderately large braided preforms or soft-body robots, and robustness will be crucial for deployment of on-site construction automation for braided composites or formwork.

Carbon fibre or glass fibre placement in multi-material systems is a common process in industrial manufacturing. Across aerospace, automotive, and other engineering ap-

Grand challenges of robotics

SCIENCE ROBOTICS | PERSPECTIVE

ROBOTICS

The grand challenges of *Science Robotics*

Guang-Zhong Yang,^{1*} Jim Bellingham,² Pierre E. Dupont,³ Peer Fischer,^{4,5} Luciano Floridi,^{6,7,8,9,10} Robert Full,¹¹ Neil Jacobstein,^{12,13} Vijay Kumar,¹⁴ Marcia McNutt,¹⁵ Robert Merrifield,¹ Bradley J. Nelson,¹⁶ Brian Scassellati,^{17,18} Mariarosaria Taddeo,^{7,8,9} Russell Taylor,¹⁹ Manuela Veloso,²⁰ Zhong Lin Wang,²¹ Robert Wood^{22,23}

Copyright © 2018
The Authors, some
rights reserved;
exclusive licensee
American Association
for the Advancement
of Science. No claim
to original U.S.
Government Works

(ii) **Biohybrid and bioinspired robots** that translate fundamental biological principles into engineering design rules or integrate living components into synthetic structures to create robots that perform like natural systems.

(iv) **Robot swarms** that allow simpler, less expensive, modular units to be reconfigured into a team depending on the task that needs to be performed while being as effective as a larger, task-specific, monolithic robot.

<http://robotics.sciencemag.org/content/robotics/3/14/eaar7650.full.pdf>